



# 基于 STM32MP1 的鸿蒙最小系统移植

马后权, 施华君

(中国电子科技集团公司 第三十二研究所, 上海 201800)

**摘要:** 鸿蒙操作系统作为国产操作系统的代表发展速度迅猛, 在嵌入式领域基于鸿蒙系统的应用也越来越广泛。本文是基于 Arm Cortex—A7 架构的鸿蒙最小系统移植, 让我们对鸿蒙系统有一个初步的认识, 为国产操作系统完善和在其他场景中的应用奠定基础。

**关键词:** 鸿蒙操作系统; 驱动; 串口; 移植; STM32MP157

**中图分类号:** TP316.9

**文献标识码:** A

## Transplantation of Harmony Minimum System Based on STM32MP1

Ma Houquan, Shi Huajun

(No.32 Institute of CETC, Shanghai 201800, China)

**Abstract:** As a domestic operating system, Harmony system is developing rapidly. There are more and more applications based on Harmony system in the embedded field. This paper is the porting of Harmony minimum system based on Arm Cortex—A7 architecture, which gives us a preliminary understanding of Harmony system. It lays a foundation for the improvement of domestic operating system and its application in other scenarios.

**Keywords:** HarmonyOS; driver; serial port; porting; STM32MP157

## 0 引言

随着智能设备的广泛应用, 近年来嵌入式技术有了长足的发展, 在嵌入式操作系统中, 国外知名的系统有 FreeRTOS、 $\mu$ C/OS、RTX 等, 它们都各有特色, 在嵌入式领域占据着重要的地位, 但自主可控且具有一定影响力的国产系统屈指可数。随着国际科研环境的变化, 发展具有自主知识产权的操作系统成为了共识, 国产嵌入式操作系统发展的一个重要方向是打造航空航天、工业装备和轨道交通、通信设备和汽车电子的自主可控操作系统<sup>[1]</sup>。近期发布的鸿蒙操作系统在国产化的道路上发展迅猛, 但是很多地方还有局限, 目前 HarmonyOS 源码支持海思的几款芯片, 为了让国产操作系统可以应用在更多的场景下, 支持市场占有率较高的芯片是一条必经之路。本文是基于 STM32MP1 的最小鸿蒙系统移植, STM32MP1 是意法半导体(ST)公司产品, 也是该公司向高端市场进军的产品, 将来会有更加广泛的应用, 基于该芯片的系统移植可以推动 HarmonyOS 生态环境的建设。

## 1 移植概述

### 1.1 鸿蒙架构

鸿蒙 OS 是一个跨终端的操作系统, 实现了模块化耦

合, 对不同的环境可以灵活地部署, 系统架构大致可以分为三层: 内核层、基础服务层、应用框架层, 可用于手机、平板、PC、汽车等终端设备上<sup>[2]</sup>。它是一套完整的操作系统, 支持多种内核, 如 Linux、LiteOS 等, Linux 与 LiteOS—a 的驱动程序是类似的, 但 LiteOS—a 的更加精简。

### 1.2 移植工作概述

**串口:** 鸿蒙串口输出时采用查询的方式, 输入采用中断, 串口可以用来打印, 也可以为应用程序提供数据, 最小系统移植只需实现串口的打印功能。

**MMU:** 单片机大多是单任务的, CPU 可以直接访问内存和外设寄存器, 对于多任务的操作系统, 需要权限管理和地址映射, 地址映射需要硬件和操作系统同时配合, 现代通用处理器使用内存管理单元(MMU)在硬件上进行支持<sup>[3]</sup>。

**中断子系统:** 根据 SystemCounter 的频率设置中断周期实现系统时钟中断, 满足多任务的交替执行和串口接收数据的中断。

**存储设备驱动和根文件:** 开发板一般有 EMMC、SD/TF 卡的驱动程序, 这些驱动设计内容较多且相对复杂, 用内存模拟既可避免复杂的驱动开发, 又可以快速实现系统移植。

## 2 LiteOS - a 的编译系统

### 2.1 Makefile 分析

编译系统涉及很多库文件和子目录,在处理函数依赖和编译选项时,逻辑清晰的 Makefile 对编译具有重要作用,以 kernel/liteos\_a/fs/fat/Makefile 为例。

如图 1 所示 Makefile 中第一行 config.mk 包含一些预先定义的变量,比如内核编译选项等,为了实现具体的目标可以在其中添加本地的编译选项。MODULE\_NAME 一般等于当前目录的名字,比如本文件所在目录是 fat,编译就会得到 libfat.a。下面两行代码中 LOCAL\_SRCS 指定了本地的源文件和第三方库中的 C 文件,这就是要编译的源文件。这些 C 文件的头文件位置由 LOCAL\_INCLUDE 来指定,LOCAL\_FLAGS 指定了头文件目录 LOCAL\_INCLUDE 和本地的编译选项 LITEOS\_GCOV\_OPTS,本地编译选项和总体编译选项(config.mk)共同决定编译要求,具体编译和链接由最后一行包含的文件 MODULE 实现,将 include \$(MODULE) 展开,具体内容是:MODULE=\$(MK\_PATH)/module.mk # kernel/liteos\_a/tools/build/mk/module.mk。分析 module.mk,编译文件一般是为了生成第一个目标,module.mk 中的第一个目标是 all,all 依赖于 LIB,由 LIB:=\$(LIBA) \$(LIBSO),需要得到后面的依赖 LIBA(LIBSO 用不上)。

```
include $(LITEOSTOPDIR)/config.mk
MODULE_NAME := $(notdir $(shell pwd))

LOCAL_SRCS := $(wildcard *.c)
LOCAL_SRCS += $(wildcard $(LITEOSTHIRDPARTY)/FatFs/source/*.c)

LOCAL_INCLUDE := \
    -I $(LITEOSTHIRDPARTY)/FatFs/source \
    -I $(LITEOSTOPDIR)/fs/fat/virpart/include
LOCAL_FLAGS := $(LOCAL_INCLUDE) $(LITEOS_GCOV_OPTS)
include $(MODULE)
```

图 1 Makefile 中的编译例子

LIBA:=\$(OUT)/lib/lib \$(MODULE\_NAME).a,也就是说 LIBA:=\$(OUT)/lib/libfat.a,接下来的核心工作是生成 libfat.a,libfat.a 生成依赖图如图 2 所示。

```
$(LIBA): $(LOCAL_OBJS)
$(HIDE)$(OBJ_MKDIR)
ifeq ($(OS), Linux)
$(HIDE)$(AR) $(ARFLAGS) $@ $(LOCAL_OBJS)
else
ifeq ($(LOCAL_MODULES),)
$(HIDE)$(AR) $(ARFLAGS) $@ $(LOCAL_OBJS)
else
$(HIDE)for i in $(LOCAL_MODULES); do \
    pushd $(OBJOUT)/$$i 1>/dev/null; \
    $(AR) $(ARFLAGS) $@ *.o; \
    popd 1>/dev/null; \
done
endif
endif
```

图 2 libfat.a 生成依赖图

LIBA 文件依赖于 OBJs 文件,系统是 LiteOS-a,走的是第二个 ifeq 分支,AR 命令将 OBJs 内一系列文件打包成 libfat.a 文件,而 LOCAL\_OBJS 的产生由命令行:LOCAL\_OBJS:=\$(LOCAL\_COBJS) \$(LOCAL\_CPPOBJS) \$(LOCAL\_ASMOBJS) \$(LOCAL\_ASMOBJS2) \$(LOCAL\_CCOBJS) 所使用的 LOCAL\_COBJS 是将以 C 为后缀的文件编译成 O 为后缀的文件,它的依赖项在 Makefile 中也可以找到。LOCAL\_COBJS:=\$(patsubst %.c, \$(OBJOUT)/%.o, \$(LOCAL\_CSRC)) 这些是执行一个 Makefile 的过程,实际编译中有许多的子目录需要执行,具体编译子目录可以从 kernel/liteos\_a/Makefile 开始分析,也可以给内核加上补丁,编译通过后查看链接命令找到所涉及的库和文件。

### 2.2 顶层 Makefile 分析

顶层 Makefile 是指在 kernel 目录下的 Makefile 文件,当执行指令:OUT=\$(LITEOSTOPDIR)/out/\$(LITEOS\_PLATFORM)时,会在 out 目录下创建新目录,不同的芯片目录名字不一样,以后在该芯片基础上编译生成的库都在这个目录下。kernel/liteos\_a/Makefile 作为顶层的 Makefile,还会包含很多文件,比如 kernel/liteos\_a/tools/build/mk/los\_config.mk,会看到 LIB\_SUBDIRS 和 LITEOS\_BASELIB 这两个变量,将需要编译的目录放在 LIB\_SUBDIRS 中,在编译子目录时进入 LIB\_SUBDIRS 指定的目录,执行 make all 指令。将 fs/fat 的目录放入左边的变量,对子目录进行编译,生成 libfat.a 文件,如下代码让内核支持 FAT 文件系统:

```
ifeq ($(LOSCFG_FS_FAT),y)
    LITEOS_BASELIB += -lfat
    LIB_SUBDIRS += fs/fat
    LITEOS_FAT_INCLUDE += -I $(LITEOSTHIRDPARTY)/FatFs/source
endif
```

同时将需要链接进内核的库放在 LITEOS\_BASELIB。在配置内核时,配置信息放在.config 里,链接内核时把 LITEOS\_BASELIB 指定的库都链接进内核。因此要增加模块时,一般在 kernel/liteos\_a/Makefile 包含的文件中增加如下 2 个变量:LIB\_SUBDIRS += 源码目录/xxx 和 LITEOS\_BASELIB += -lxxx,在源码目录/xxx 下仿照 fs/fat/Makefile 增加 Makefile,最终保证顶层 Makefile 文件能看到就成功了。

## 3 系统移植

### 3.1 添加单板

移植操作系统最基本的是让系统支持相关芯片,能在 Board 页面切换,添加新的单板的方式可以仿照海思芯片



时,都是跳转去执行具体串口设备驱动程序的函数。第三层是硬件驱动程序,它分为两部分:device\_t 和 driver\_t。在 device\_t 中设置资源,比如寄存器物理基地址、中断号等。device\_t 示例代码如图 5 所示。

```
// callback never be null pointer
static void uart_add_device(add_res_callback_t callback)
{
    device_t uart_dev;
    UART_ADD_DEVICE(uart_dev, 0);
    callback("uart", SYS_RES_MEMORY, 0, UART4_REG_PBASE,
            UART4_REG_PBASE + UART_IOMEM_COUNT, UART_IOMEM_COUNT);
    callback("uart", SYS_RES_IRQ, 0, NUM_HAL_INTERRUPT_UART4,
            NUM_HAL_INTERRUPT_UART4, 1);
}
//endif
```

图 5 device\_t 示例代码

在 driver\_t 中提供函数,比如 device\_probe 和 device\_attach,当内核发现有名字系统的 device\_t、driver\_t 时,就会调用 driver\_t 中的 device\_probe、device\_attach 函数,根据 device\_t 得到资源,注册驱动 register\_driver。driver\_t 示例代码如图 6 所示。

```
static device_method_t uart_methods[] =
{
    /* Device interface */
    DEVMETHOD(device_probe, stm32mp157_probe),
    DEVMETHOD(device_attach, stm32mp157_attach),
    DEVMETHOD(device_detach, stm32mp157_detach),
    DEVMETHOD(device_shutdown, bus_generic_shutdown),
    DEVMETHOD_END
};

static driver_t uart_driver =
{
    .name = "uart",
    .methods = uart_methods,
    .size = sizeof(struct uart_softc),
};
```

图 6 driver\_t 示例代码

这种方式将操作函数、资源分离,以后想换一个硬件,只需要修改 device\_t,而 driver\_t 保存不变。第四层的结构体函数提供串口操作,比如构造结构体:

```
static const struct file_operations_vfs g_helloDevOps = {
    .open = hello_open,
    .close = hello_close,
    .read = hello_read,
};
```

将构造好的结构体注册到内核:

```
int ret = register_driver("/dev/hello", &g_helloDevOps, 0666,
NULL);
```

注册驱动时提供文件名,以后通过文件名和驱动建立联系,应用程序在打开文件使用驱动时,Linux 和 LiteOS-a 在 APP 层面都一样,打开文件读和写。很多工作 u-boot 已经做好,我们只需要串口能实现单个字符,注册出口接收中断函数确定中断号、使能中断、在中断函数中读取数据,只需要实现 startup(确定中断号)、request\_irq(使能中断、提供中断处理函数)、start\_tx(发送字符串)即可,其他

函数可以设为空。

### 3.4 时钟系统

在操作系统中,需要一个系统时钟,各类芯片都有自己的定时器,它们的编程方法互不相同,Generic Timer 是 ARM 的一种硬件实现,可以实现统一的编程方法。Generic Timer 分为两部分:共享的 System Counter 和各个 Processor 专有的 Timer。System Counter 是绝大部分部件工作的时钟源。时钟系统可以选择为 PLL 输出、HSI 输出或者 HSE 输出。HSI 和 HSE 可以通过分频加至 PLLSRC,并由 PLLMUL 进行倍频后经选择作为 Generic Timer<sup>[4]</sup>。Generic Timer 有两种访问方式:一种是 CP15 协处理器命令,当 Processor 去访问它时可以使用 CP15 协处理器命令;另一种由 Memory Mapped 寄存器提供统一的时间 Timer,可以设置周期性的事件,给 Processor 提供中断信号。System Counter 是时钟源,为进入 Porcessor 中的 Timer。Porcessor 中的 Timer 产生的中断进入 GIC,作为 PPI (Private Peripheral Interrupt) 传给 Processor。System Counter 是系统级别的 (System Level),给整个系统提供时钟,可以使用 Memeory Mapped 的寄存器来访问。每个 Processor 里都有一个 Timer,这些 Timer 可以发出周期性的中断,给 Processor 提供系统时钟。Timer 需要使用 CP15 协处理器命令来访问。

时钟源和设置/启动 System Counter:一般 u-boot 里做好了,对于 Processor 的 Timer 的设置,LiteOS-a 里面也有完善的代码,在进行系统移植需要修改时钟系统时,可以按照图 7 的流程修改。

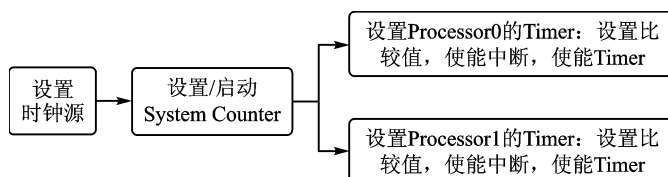


图 7 时钟系统内图

### 3.5 存储和根文件系统

在嵌入式设备中存储模块是不可或缺的部分,闪存体积小且存取速度快,适合作为嵌入式设备的存储设备。目前市场有两种非易失的存储器:Nor Flash 和 Nand Flash<sup>[5]</sup>,这些 Flash 又有不同的接口,比如 SPI 接口等。不同 Flash 的访问方法各有不同,但一般都有读、写、擦除这些操作,由此抽象出一个软件层:MTD (Memory Technology Device)。它封装了不同 Flash 的操作,主要是为了使新的 memory 设备的驱动更加简单,为此它在硬件和上层之间提供了一个抽象的接口。不同的 Flash 要提供它自己的 MtdDev 结构体,如图 8 所示。

```
void ramnor_register(struct MtdDev *mtd)
{
    //mtd->priv = (void *)DDR_RAMFS_VBASE;

    //mtd->size = DDR_RAMFS_SIZE;
    mtd->eraseSize = 0x10000;

    mtd->type = MTD_NORFLASH;

    mtd->erase = ramnor_erase;
    mtd->read = ramnor_read;
    mtd->write = ramnor_write;
}
```

图8 MtdDev 结构体实现图

使用内存模拟 Flash 时,还需要从内存中指定一块,确定其大小和地址,在先前添加开发板创建的文件夹下的 driver/mtd/spi\_nor/src/common/spinor.c 文件里面的 spinor\_init(void) 函数中设置。内存设置好后需要设置根文件系统,要构建一个可用的根文件系统可以按照 FHS (File-system Hierarchy Standard) 的标准布局文件目录,创建必要的库文件和二进制文件<sup>[6]</sup>。库/lib,比如 printf 函数就是在库里的;二进制文件/bin,hello 这样的程序放在/bin 或/usr/bin 这些目录里,文件系统至少有这些二进制文件;init,内核启动的第一个 APP,它会去启动其他 APP,比如 shell。shell 也是一个 APP,可以让我们输入各类命令,还有自己的 APP:比如输出程序 hello.c,当/etc 想自动启动 APP 时,init 进程根据配置文件去启动其他 APP,比如/etc/init.cfg。/dev 为设备节点,LiteOS-a 中不需要我们自己创建。在 kernel/liteos\_a 目录执行 make help。根文件制作命令图如图 9 所示。

```
1.====make help:    get help information of make
2.====make:         make a debug version based the .config
3.====make debug:   make a debug version based the .config
4.====make release: make a release version for all platform
5.====make release PLATFORM=xxx: make a release version only for platform xxx
6.====make rootfsdir: make a original rootfs dir
7.====make rootfs FSTYPE=***: make a original rootfs img
8.====make test: make the test results app and put it into the rootfs dir
9.====make test_apps FSTYPE=***: make a rootfs img with the test results app in it
xxx should be one of (hl3516cv300 hl3516ev200 hl3556av100/cortex-a53_arch32 hl3559a
v100/cortex-a53_arch64)
*** should be one of (jffs2)
```

图9 根文件制作命令图

图 9 中,make rootfs 指令用来制作根文件系统,在根文件的基础上建立文件夹去填充,文件系统制作完成后,将其制作成镜像文件。

#### 4 移植结果和测试

用串口工具打开 serial,使用 STM32CubeProgrammer 将其烧进内存中,按照提示进入鸿蒙系统,输入指令 help,系统

正常工作,移植成功。鸿蒙串口输入指令如图 10 所示。

```
LiteOS # help
*****shell commands*****
arp          cat          cd          chgrp       chmod        chown        cp
date         cpup         dhclient    dmesg       dns          format       free         help
ifconfig     hwi          ipdebug     kill         log          ls           lsfd         memcheck
mount        mkdir        netstat     oom          partinfo     partition    ping         ping6
pwd          pmm          readreg     rm           rmdir        sem          stack        statfs
swtar        su           sync         systeminfo   task         telnet       tftp         touch
uname        umount       v2p         virstatfs    vim          watch        writeproc
LiteOS #
```

图10 在鸿蒙串口输入指令

## 5 结 语

本文是基于 STM32MP157 芯片的鸿蒙操作系统移植,让大家对鸿蒙系统有个初步的了解,由于是最小系统移植,许多功能还需要完善,在使用系统时可以根据实际需要修改。国产操作系统正在向好的方向发展,以后的应用场景也会越来越多,鸿蒙 OS 会在国产操作系统发展的道路上做出自己的贡献。

#### 参考文献

- [1] 何小庆.国产嵌入式操作系统发展思考[J].单片机嵌入式系统应用,2019(12):4-10.
- [2] 谢克强.鸿蒙操作系统打造生态的路径思考[J].单片机与嵌入式系统应用,2019(10):3-6.
- [3] 党倩,杨婷.关于 CKCORE 嵌入式内存管理单元的研究和解析[J].微处理器与可编程控制器,2014(11):52-54.
- [4] 荆海霞.STM32 系列微控制器的时钟系统分析[J].科技信息,2008(33):517-518.
- [5] 苏缨墩,钟汉如.嵌入式 Linux 中的 Nand Flash 驱动详解[J].工业仪表与自动化装置,2011(4):51-60.
- [6] 茅胜荣,肖家文,乔东海.用 SD 卡定制嵌入式 Linux 系统的最小系统[J].单片机与嵌入式系统应用,2017(10):22-26.

马后权(硕士研究生),主要研究方向为嵌入式系统开发和应用;施华君(工程师),主要研究方向为嵌入式计算技术、计算机仿真技术、自主可控计算技术。通信作者:施华君,shi\_huajun@yeah.net。

(责任编辑:薛士然 收修改稿日期:2021-10-19)

42 [10] 杜瑞超,华继学,翟夕阳,等.基于改进 Real AdaBoost 算法的软件可靠性预测[J].空军工程大学学报(自然科学版),2018,19(1):91-96.

[11] Jabeen G, Luo P, Afzal W. An improved software reliability prediction model by using high precision error iterative analysis method[J]. Software Testing, Verification and Reliability, 2019, 29(6): 97-108.

[12] 元慧,史颖,李灯熬,等.基于连续型深度置信神经网络的软件可靠性预测[J].计算机科学,2021,48(5):86-90.

张博云(助理工程师),主要研究方向为基于人工智能的软件可靠性预测、软件性能测评、嵌入式软件开发等。通信作者:张博云,1084693991@qq.com。

(责任编辑:薛士然 收稿日期:2021-06-02)